

Prompt Engineering Practices Guide

Concrete patterns used while building `health\_agents`, grounded in the actual prompts and code in this repo (`src/health\_agents/prompts/`) - not general theory. Each entry says what we did, where, and why it mattered in practice.

1. Give the model a strict output contract, and always parse defensively

Every LLM call that feeds a decision back into code asks for one exact format and nothing else:

- Guardrail (`prompts/guardrail.py`): a single-line JSON object, `{"passed": bool, "message": str}` - no markdown fences, no prose.
- Classifier (`prompts/classifier.py`): exactly one lowercase word.
- Interview (`prompts/interview.py`): single-line JSON, `{"sufficient": bool, "question": str}`.

The prompt asking for a strict format is only half the contract - the code side (`\_parse\_decision`, `\_parse\_verdict` in the graph modules) never trusts the model to actually comply. It strips markdown fences defensively, and always has a fallback for malformed output. Models occasionally add a stray sentence or wrap JSON in ````json```` fences even when told not to; plan for that from the start rather than patching it after a crash.

2. Separate data from instructions, explicitly, with delimiters

Anywhere user- or document-supplied text enters a prompt, it's wrapped in an explicit tag and the system prompt states in plain language that content inside is **data to process**, never **instructions to follow** - even if it's phrased as a command, a role-change request, or a request to reveal the system prompt:

- Summarization wraps the report in `<health_report>...</health_report>` (`graphs/summarization\_graph.py`).
- Synthesis treats the full interview transcript the same way (`graphs/synthesis\_graph.py`).
- QnA's own system prompt tells it to keep discussing health even if the user tries to override its role (`prompts/qna.py`).

This is the concrete, testable defense against prompt injection embedded in a document or a chat message. It's also literally testable: our tests

record the exact ``messages`` list sent to a fake LLM and assert the delimiter and framing are present (see ``test_summarize_wraps_document_as_untrusted_data``, ``test_synthesize_sends_transcript_as_untrusted_data``).

3. Enforce hard constraints in code, not in the prompt alone

The interview agent must ask at least 6 and at most 10 questions. The prompt *asks* the model to respect that range, but the graph node (``graphs/interview_graph.py``) enforces it directly: it never even calls the LLM once the max is hit, and it overrides ``sufficient=false`` if the model claims "enough" before the minimum. An LLM's adherence to a numeric constraint in a prompt is a preference, not a guarantee - if a wrong answer would break something (an infinite loop, a rule you actually need to hold), enforce it in code and let the prompt handle the part that's genuinely a judgment call (*which* question to ask next).

4. Choose fail-open vs. fail-closed deliberately, per task

When a parse fails, the safe default is different depending on what's actually risky:

- Guardrail parse failure -> **fails closed** (``passed=False``). We can't confirm the query is safe, so don't let it through unverified.
- Classifier parse failure -> **defaults to ``qna``**. An unparseable routing decision degrading into "just answer the question" is the least likely outcome to do something the user didn't ask for - safer than defaulting to, say, generating a PDF nobody asked for.

Both are "safe defaults," but safety points in opposite directions depending on the task. Decide this explicitly rather than picking one fallback pattern and reusing it everywhere.

5. State the actual decision rule, not just the category labels

The first version of the classifier prompt defined categories ("summarize" = has an existing report; "synthesize" = wants a new one) but a query like *"I want a summary of what tests I should track for my visit"* still got misrouted to ``qna`` - the word "summary" pattern-matched toward ``summarize`` even with no document involved. The fix wasn't more categories, it was naming the actual test explicitly: *"the key question is whether a document already exists - not whether the word 'summary' appears"* (``prompts/classifier.py``). Small/fast models lean on surface wording; if there's a wording trap, say so directly in the prompt.

6. Add worked examples for the cases that are actually ambiguous

After finding the misroute above, we added a short list of input -> expected-output examples to the classifier prompt, including the exact phrasing that broke - not generic examples, the specific edge case that failed. Few-shot examples targeted at real failure modes are far higher-leverage than generic ones "just in case."

7. Reach for a task-tuned model when one exists

When picking the guardrail's model, we checked what was actually available on the account (`client.models.list()`) and found `openai/gpt-oss-safeguard-20b` - a model tuned specifically for safety/policy classification - sitting alongside the general-purpose chat models. A specialized model for a specialized decision beats a generic chat model prompted to behave like one, when the option exists.

8. Match model size to what the task actually demands, not one default everywhere

Five independently configurable models (`config.py`), split by what the call site needs, not habit:

- Guardrail / classifier: single-shot, low-complexity, run on *every* query -> smaller/faster model.
- Summarize / synthesize / QnA: real reasoning over clinical content, open-ended generation -> larger model.

`get_groq_chat(model, ...)` makes `model` a required argument on purpose, so picking a model is a visible decision at every call site, not an accidentally-shared implicit default.

9. Give the model explicit instructions for edge-of-conversation states

QnA needs to greet on the very first turn and never again. Rather than special-casing that in code, the system prompt states the rule directly: `"if this is the very start of the conversation (no prior user message), open with a brief greeting... do not repeat this greeting on later turns"` (`prompts/qna.py`). The graph node stays a single, uniform "respond to the conversation so far" call regardless of turn number - the state-dependent behavior lives entirely in the prompt.

10. Design the failure path, not just the happy path

The interview decision parser has an explicit fallback question (`FALLBACK_QUESTION = "Can you tell me more about your symptoms?"` in `graphs/interview_graph.py`) for when the model's output can't be

parsed as JSON. A conversational flow that goes blank or crashes on a malformed response is a worse failure than asking a slightly generic follow-up question. Decide what "safe but unhelpful" looks like for each prompt, and make it the fallback.

11. Unit tests catch wiring bugs; only the live model catches prompt bugs

All 27 automated tests use `langchain\_core`'s fake chat models - they verify code correctness (parsing, state wiring, delimiter placement) with no API key and no network cost, and that's the right scope for a test suite you run on every change. But they cannot catch a prompt that's *correctly wired* and *wrong* - the classifier misroute in section 5 only showed up by hand-building a small accuracy check (a table of realistic inputs and expected outputs) and running it against the real model. Keep both: fast fake-model tests for every change, and a live accuracy spot-check whenever a prompt's wording actually changes.